

Select CLKDV output and select 10 in the Divide by Value drop-down box (Fig. 9).

15. Click Next, Next and Finish to close the wizard.

16. DCM_Divide.xaw is added to the list of project source files as shown in Fig. 10. It contains the architecture wizard settings.

17. Select DCM_Divide.xaw. In

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download at source.elymag.com



Fig. 10: Adding DCM_Divide.xaw

```
COMPONENT DCM_Divide
PORT
CLKIN_IN : IN std_logic
RST_IN : IN std_logic
CLKDV_OUT : OUT std_logic
CLKIN_INFUG_OUT : OUT std_logic
CLK0_OUT : OUT std_logic
LOCKED_OUT : OUT std_logic
END COMPONENT;
```

Fig. 11: Component declaration

```
Inst_DCM_Divide: DCM_Divide PORT MAP(
CLKIN_IN => ,
RST_IN => ,
CLKDV_OUT => ,
CLKIN_INFUG_OUT => ,
CLK0_OUT => ,
LOCKED_OUT => )
);
```

Fig. 12: Component instantiation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_FUNCTIONS.ALL;
use IEEE.NUMERIC_STD.ALL;

-- Document the following library declaration is unnecessary
-- library IEEE;
-- use IEEE.NumericStd.ALL;

entity dcm_div
Port
CLKIN_IN : IN STD_LOGIC;
RST : IN STD_LOGIC;
CLKDV_OUT : OUT STD_LOGIC;
CLKIN_INFUG_OUT : OUT STD_LOGIC;
CLK0_OUT : OUT STD_LOGIC;
LOCKED_OUT : OUT STD_LOGIC;
end entity;

architecture Behavioral of dcm_div is
COMPONENT DCM_Divide
PORT
CLKIN_IN : IN std_logic;
RST_IN : IN std_logic;
CLKDV_OUT : OUT std_logic;
CLKIN_INFUG_OUT : OUT std_logic;
CLK0_OUT : OUT std_logic;
LOCKED_OUT : OUT std_logic;
END COMPONENT;

begin
end Behavioral;
```

Fig. 13: Clockdiv.vhd

the Processes tab for sources window, double click View HDL Instantiation Template. An HDL implementation file DCM_Divide.vh opens. It

comprises two items, namely, component declaration (Fig. 11) and component instantiation as shown in Fig. 12.

18. Copy the component declaration template and paste it in the Clock-Divider.vhd file in the space between 'architecture Behavioral of dcm' and 'begin' statement as shown in Fig. 13.

19. Copy the component instantiation template and paste it in the Clock-Divider.vhd file after the 'begin' statement as shown in Fig. 14.

20. Make the changes in the instantiation code as given below:

```
Inst_DCM_Divide : DCM_Divide PORT
MAP(
```

```
CLKIN_IN => CLOCK_50MHz,
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_FUNCTIONS.ALL;
use IEEE.NUMERIC_STD.ALL;

-- Document the following library declaration is unnecessary
-- library IEEE;
-- use IEEE.NumericStd.ALL;

entity dcm_div
Port
CLKIN_IN : IN STD_LOGIC;
RST : IN STD_LOGIC;
CLKDV_OUT : OUT STD_LOGIC;
CLKIN_INFUG_OUT : OUT STD_LOGIC;
CLK0_OUT : OUT STD_LOGIC;
LOCKED_OUT : OUT STD_LOGIC;
end entity;

architecture Behavioral of dcm_div is
COMPONENT DCM_Divide
PORT
CLKIN_IN : IN std_logic;
RST_IN : IN std_logic;
CLKDV_OUT : OUT std_logic;
CLKIN_INFUG_OUT : OUT std_logic;
CLK0_OUT : OUT std_logic;
LOCKED_OUT : OUT std_logic;
END COMPONENT;

begin
Inst_DCM_Divide: DCM_Divide PORT MAP(
CLKIN_IN => ,
RST_IN => ,
CLKDV_OUT => ,
CLKIN_INFUG_OUT => ,
CLK0_OUT => ,
LOCKED_OUT => )
);
end Behavioral;
```

Fig. 14: ClockDivider with component instantiation

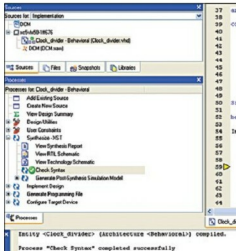


Fig. 15: Final output

```
RST_IN => RST,
CLKDV_OUT => CLOCK_100kHz,
CLKIN_INFUG_OUT => open,
CLK0_OUT => CLK0 ,
LOCKED_OUT => LOCK
);
```

The final output of ClockDivider when compiled should give a "...completed successfully" message as shown in Fig. 15.

21. In our example, we are connecting the input clock CLOCK_50MHz directly to the DCM and taking the divided clock output CLKDV_OUT as the final output CLOCK_100kHz. After compilation, load the code into the device. The CLOCK_100kHz port generates 100kHz output, which you can verify from the oscilloscope. This signal can be used anywhere as required in the program.

In case one needs to use the inputs and outputs of the DCM internally, they have to be declared as signals. To use CLK0_OUT clock output internally, it is portmapped to CLK0 which is declared a signal as shown below:

```
"signal CLK0: std_logic;"
```

Conclusion

This part described how to generate a clock of the desired frequency using DCMs in FPGAs. The concluding part will talk about embedded processor design using FPGAs.

To be continued...

Varsha Agrawal is a scientist at Laser Science and Technology Center (LASTEC), a premier DRDO lab working in the field of laser-based defence systems. She has more than 13 years of R&D experience in the design and development of a variety of systems for defence-related applications. She has authored two books and published more than 20 research papers and technical articles